

Work Report

Prepared by: Adhithyan P.B

Role: Flutter developer

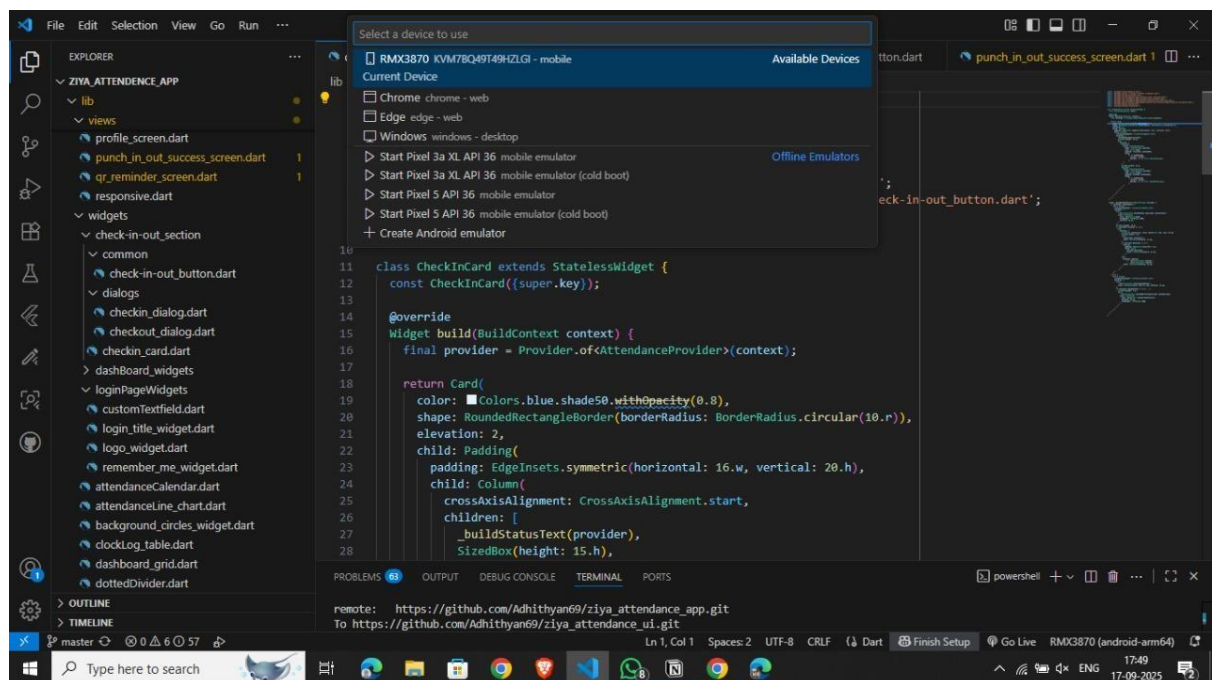
Date: 17-09-2025

1. Flutter Setup

I successfully completed the installation and setup of the Flutter development environment. The following tools were installed and configured on the system:

- **Flutter SDK** – for mobile application development.
- **Android Studio** – for emulator, debugging, and development support.
- **Visual Studio Code** – as the main IDE for coding Flutter applications.

During the setup, I ensured proper device configuration for smooth development and testing. Due to emulator limitations, I configured a physical device for better performance and stability.



2. Screens Created (with MVVM + Responsive Implementation)

As part of the project development, I created and implemented the following application screens. The project follows the **MVVM (Model-View-ViewModel)** architecture, ensuring **clean code separation**,

better maintainability, and scalability. The architecture is still being improved as new modules are added.

Additionally, I have started integrating **Flutter ScreenUtil package** across all screens to ensure **responsiveness**. This makes the app adaptable to different screen sizes (mobiles, tablets, large devices). The process of converting all screens to responsive layout is ongoing.

Implemented Screens:

1. Login Screen

- Implements user authentication with email/phone and password.
- Connected with ViewModel to validate inputs and handle API calls.
- Responsive UI: Widgets and font sizes adapt using ScreenUtil.

2. Forgot Password Screen

- Allows users to initiate password recovery.
- Uses ViewModel to send requests to backend for OTP generation.
- Responsive UI: Form fields and buttons adjust proportionally across screen sizes.

3. OTP Verification Screen

- Provides One-Time Password (OTP) input for secure verification.
- MVVM ensures OTP format validation before API request.
- Responsive UI: Input boxes and spacing scale dynamically.

4. Reset Password Screen

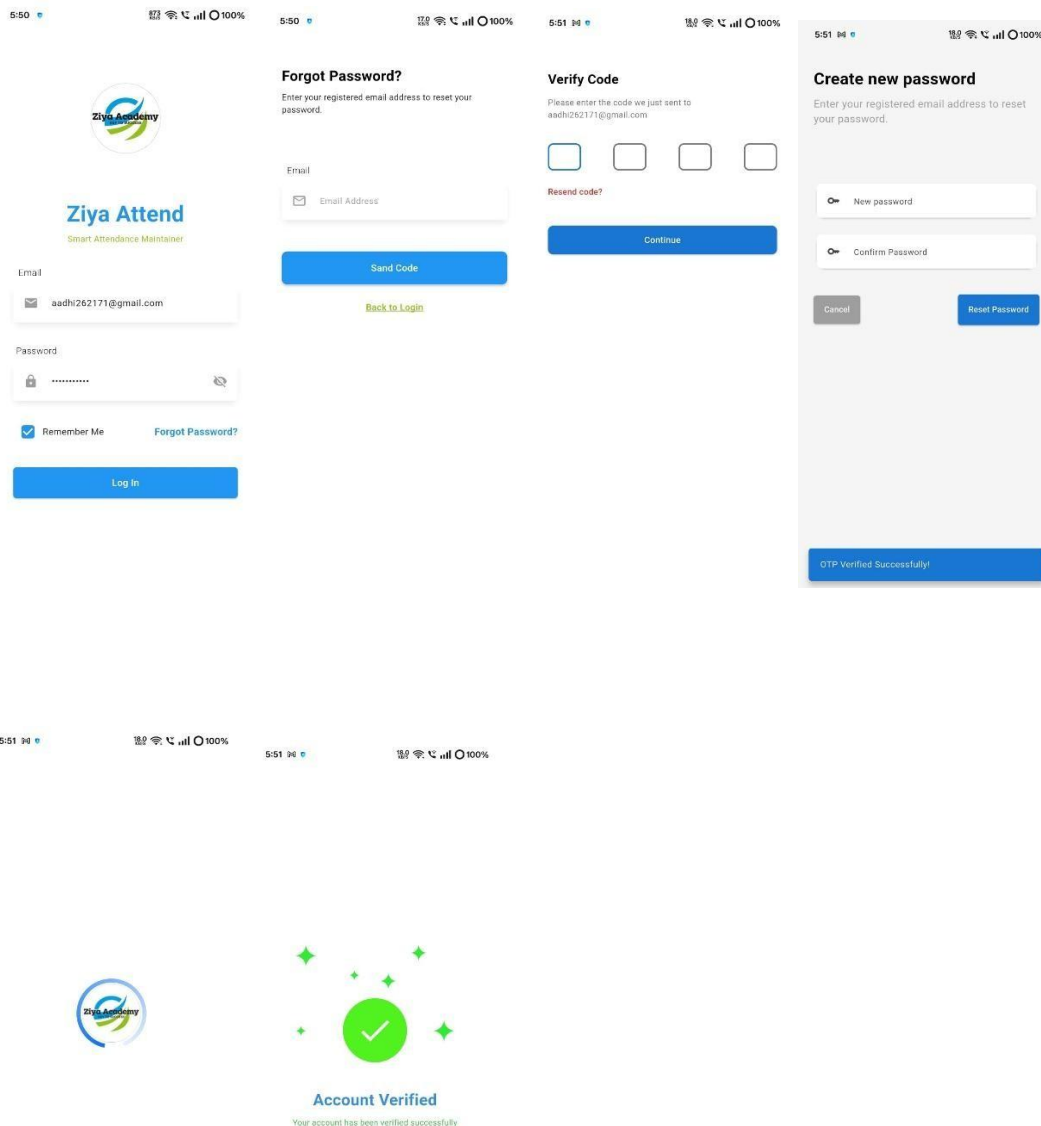
- Lets users set a new password after OTP verification.
- Uses ViewModel for password strength and confirm password validation.
- Responsive UI: Layout scales properly on both small and large devices.

5. Buffering / Loading Screen

- Displays while data is being fetched or verified.
- ViewModel manages loading states for separation of concerns.
- Responsive UI: Loader adapts its size and position based on device.

6. Account Verified Screen

- Confirms successful verification of the user's account.
- MVVM handles backend response states.
- Responsive UI: Message, icons, and spacing adjust seamlessly.



3. Additional Contribution

- I am conducting daily teaching sessions for two students, currently teaching API integration in Flutter
- I have assigned tasks for interns to practice and implement what they learn on a daily basis.

4. Next Plan

The upcoming development plan is to:

- Implement Check-In and Check-Out features for attendance/activity tracking.
- Properly integrate this with the backend to ensure secure data storage and retrieval.

- Continue converting all screens to responsive design using ScreenUtil, making the application fully device-independent.
- Keep improving the MVVM structure as new modules are added, ensuring long-term maintainability.
- Continue teaching API integration concepts to students and assigning practical tasks to interns for skill development.